Agama Finance

Technical Architecture

Private Credit Yield Vaults on Stellar

Soroban | SEP-41 | Soroswap | Etherfuse | CCTP | MoneyGram

Contents

> **REVISION NOTE · SEPTEMBER 2026**
>
> The Blend v2 integration described in earlier versions of this document has been removed, following the Comet BLND-USDC exploit and Blend's removal from the SCF Integration List, where Blend V2 is being wound down. It is not replaced by another protocol.
>
> The instant-withdrawal liquidity role Blend played is now an on-chain reserve floor, enforced by the Allocation Engine and again by the Vault itself. The floor is a **share of a base, expressed in basis points**, not a fixed sum of USDC: an allocation reverts if the call would leave the Vault holding less free USDC than `floor_bps` of that base. The base is everything the protocol holds that the withdrawal queue has no claim on — free plus deployed — **plus everything it has ever written off**, because a write-down lowers recorded exposure with no cash moving and a base the caller can lower is not a floor. It is set to **2500 bps — 25%** — on testnet.
>
> The unit is the substance of the replacement, not a detail of it. A liquidity buffer parked in a lending protocol is only as instant as that protocol's utilization on the day it is needed; USDC that never left the Vault has no such dependency. But a buffer denominated as a fixed amount stops meaning anything as the book moves — it is most of a small vault and a rounding error in a large one — and it is the *ratio* of cash to obligations that decides whether a withdrawal can be paid. Expressing the floor as a share is what makes it scale with the book it is protecting, and it is why the guarantee survives growth instead of being re-tuned by hand after it.
>
> Fast-exit liquidity is therefore a protocol parameter anyone can read on-chain, in the same units the caps are written in, not a dependency on a third party's solvency.

1. Introduction

1.1 High-Level Overview

Agama is a private-credit yield infrastructure for tokenized real-world assets, built natively on Stellar using Soroban smart contracts (Rust). Users deposit USDC into curated vaults and receive **agUSD**, a composable synthetic dollar backed by diversified credit pools. Staking agUSD produces **sagUSD**, a yield-bearing token whose value appreciates as private credit repayments and on-chain strategies generate returns.

Both agUSD and sagUSD are issued as Soroban/**SEP-41** tokens. The on-chain Allocation Engine distributes capital across vetted pools—including **Etherfuse Stablebonds** for Stellar-native government-bond exposure and off-chain private credit pools from vetted originators—while enforcing concentration caps by pool, originator, and jurisdiction.

The protocol composes existing Stellar ecosystem primitives rather than reimplementing solved problems: **Soroswap** for AMM liquidity, **CCTP** for cross-chain USDC bridging, and **MoneyGram** (SEP-24) and **Bridge** for fiat on/off-ramps. sagUSD additionally follows the share-price yield economics established by **DeFindex**, without calling its contracts. agUSD functions as a composable building block for other Soroban protocols, bringing sticky, real-yield-backed TVL to the Stellar ecosystem.

1.2 Core Components

- **Agama dApp** — Web application for deposits, staking, and portfolio transparency. Wallet connection via the Stellar Wallets Kit (Freighter, xBull, Albedo, Ledger). Bridge tab for CCTP cross-chain deposits. Swap interface via Soroswap Router API.
- **Backend API & Indexer** — Node.js services ingesting Soroban contract events for NAV history, yield accrual, portfolio analytics, transparency reporting, and settlement management.
- **Soroban Smart Contracts (Rust)** — On-chain core: Vault (deposits/withdrawals, agUSD mint/redeem), agUSD and sagUSD SEP-41 tokens, Allocation Engine (pool routing, caps, multi-adapter), and Oracle Adapter (multi-source NAV validation).
- **Allocation Engine** — Routes deposited capital across Etherfuse Stablebonds and private credit pools via a uniform adapter interface, enforcing on-chain concentration caps.
- **Oracle Integration** — Multi-source NAV pipeline: Reflector for asset prices, custom reporter for private credit NAV, Etherfuse feed for bond pricing. Staleness and deviation checks on-chain.
- **On/Off-Ramp & Cross-Chain** — MoneyGram (SEP-24) for retail fiat ramp. Bridge for institutional wires. CCTP for native USDC bridging from Ethereum/Arbitrum.
- **Liquidity** — agUSD/USDC and sagUSD/agUSD pools on Soroswap. Stellar DEX order books as secondary liquidity venue.

1.3 Definitions and Acronyms

Term	Definition
USDC	USD Coin, fiat-backed stablecoin by Circle, native on Stellar. Vault deposit asset.
Soroban	Stellar's smart contract platform. Contracts written in Rust, compiled to WASM.
SEP-41	Standard token interface for Soroban contracts. agUSD and sagUSD implement SEP-41.
agUSD	Agama's synthetic dollar, minted 1:1 against USDC deposits.
sagUSD	Staked agUSD. Yield accrues via increasing sagUSD/agUSD exchange rate.
NAV	Net Asset Value — on-chain reported value of the portfolio backing agUSD.
Allocation Engine	Soroban contract routing vault capital across pool adapters with cap enforcement.
Reserve Floor	Minimum share of `floor_base` the Vault must retain as free USDC, in basis points. Enforced by the Allocation Engine at allocation time and by the Vault at release time, against its own book. 2500 bps (25%) on testnet.
`floor_base`	The denominator of the reserve floor: net assets plus cumulative recognised losses. Equal to net assets for a protocol that has never written anything off, and deliberately larger afterwards, so that recognising a loss cannot create room to deploy.
Free reserves	Idle USDC in the Vault less what the withdrawal queue is owed. Queued claims have already burned their agUSD, so the cash behind them is a liability rather than deployable capital.
Net assets	Free reserves plus deployed capital. The denominator every cap and the floor are measured against.
Originator	Vetted private credit counterparty receiving vault allocations.
Reflector	Decentralized push-based oracle network on Stellar.
DeFindex	Yield infrastructure for Stellar. sagUSD follows the same share-price yield economics, not DeFindex's contract interface.
Soroswap	Primary AMM on Soroban. Provides agUSD liquidity pools.
CCTP	Circle Cross-Chain Transfer Protocol. Native 1:1 USDC bridge between blockchains.
Etherfuse	Stablebonds — Stellar-native tokens backed by government bonds with embedded yield.
SEP-24	Interactive deposit/withdrawal flows with anchors (used for MoneyGram).
SEP-57	T-REX — Token for Regulated Exchanges. Standard for compliance-controlled RWA tokens on Stellar.
TTL	Time To Live — Soroban storage lifetime, managed with archival thresholds and periodic bumps.

2. Architecture Overview

2.1 C1 Context

External Actors

- **LP / Depositor** — Deposits USDC into Agama vaults, receives agUSD, optionally stakes into sagUSD.
- **Asset Originator** — Private credit counterparty receiving allocations and servicing repayments.
- **Curator / Risk Committee** — Whitelists pools and originators, sets concentration caps, manages risk parameters.

External Systems

System	Role	Integration List
Stellar Network	All on-chain operations: USDC settlement, Soroban execution	—
Circle (USDC)	Issuer of native USDC on Stellar	—
Reflector Oracle	Decentralized price feeds (USDC, XLM)	—
DeFindex	Share-price yield economics referenced by sagUSD, no contract calls	Integration List
Soroswap	AMM liquidity pools + Router API	Integration List
Etherfuse	Stablebonds — Stellar-native RWA collateral	Integration List
CCTP (Circle)	Native cross-chain USDC bridge	Integration List
MoneyGram	Fiat cash on/off-ramp (SEP-24), 180+ countries	Integration List
Bridge	Institutional fiat ramp (bank wires, ACH)	—

2.2 C2 Containers

Container	Technology	Responsibility
Agama dApp	TypeScript / Next.js	User-facing: deposits, staking, swaps (Soroswap Router), bridge (CCTP), transparency dashboard. Connects wallets via Stellar Wallets Kit. Submits transactions via Soroban RPC.
Backend API & Indexer	Node.js / PostgreSQL	Event ingestion, analytics (APY, NAV, exposure), settlement management (off-chain → on-chain), oracle reporting pipeline, MoneyGram SEP-24 adapter, Bridge API integration.
Soroban Contracts	Rust → WASM	Vault, agUSD, sagUSD, Allocation Engine (with Etherfuse/private credit adapters), Oracle Adapter.

2.3 C3 Components

Agama dApp

- Wallet module — Stellar Wallets Kit for auth and transaction signing.
- Deposit/withdraw and stake/unstake interfaces — Soroban contract calls.
- Swap interface — Soroswap Router API for agUSD/USDC trading.
- Bridge tab — CCTP via Circle Bridge Kit SDK.
- Transparency dashboard — NAV, yield, pool exposure from Backend API.

Backend API & Indexer

- **Event Ingestion Service** — Soroban events → PostgreSQL.
- **Analytics Service** — APY, share price history, concentration metrics.
- **Settlement Manager** — Bridges off-chain credit repayments back on-chain.
- **Originator Reporting Adapter** — Collects off-chain servicing data, reconciles with on-chain NAV.
- **On/Off-Ramp Adapters** — MoneyGram SEP-24 + Bridge REST API.

Soroban Smart Contracts

- **Vault Contract** — USDC deposits, agUSD mint/burn, FIFO withdrawal queue.
- **agUSD Token (SEP-41)** — `mint` restricted to the Vault. `burn` and `burn_from` are the standard SEP-41 holder-authorized paths.
- **sagUSD Staking Contract** — Share-based vault on DeFindex's yield economics, not its interface. Yield via exchange rate appreciation. Two-step unstake behind a cooldown.
- **Allocation Engine** — Pool routing with adapters (Etherfuse, private credit). Concentration caps.
- **Oracle Adapter** — Multi-source NAV validation (Reflector, custom reporter, Etherfuse feed).

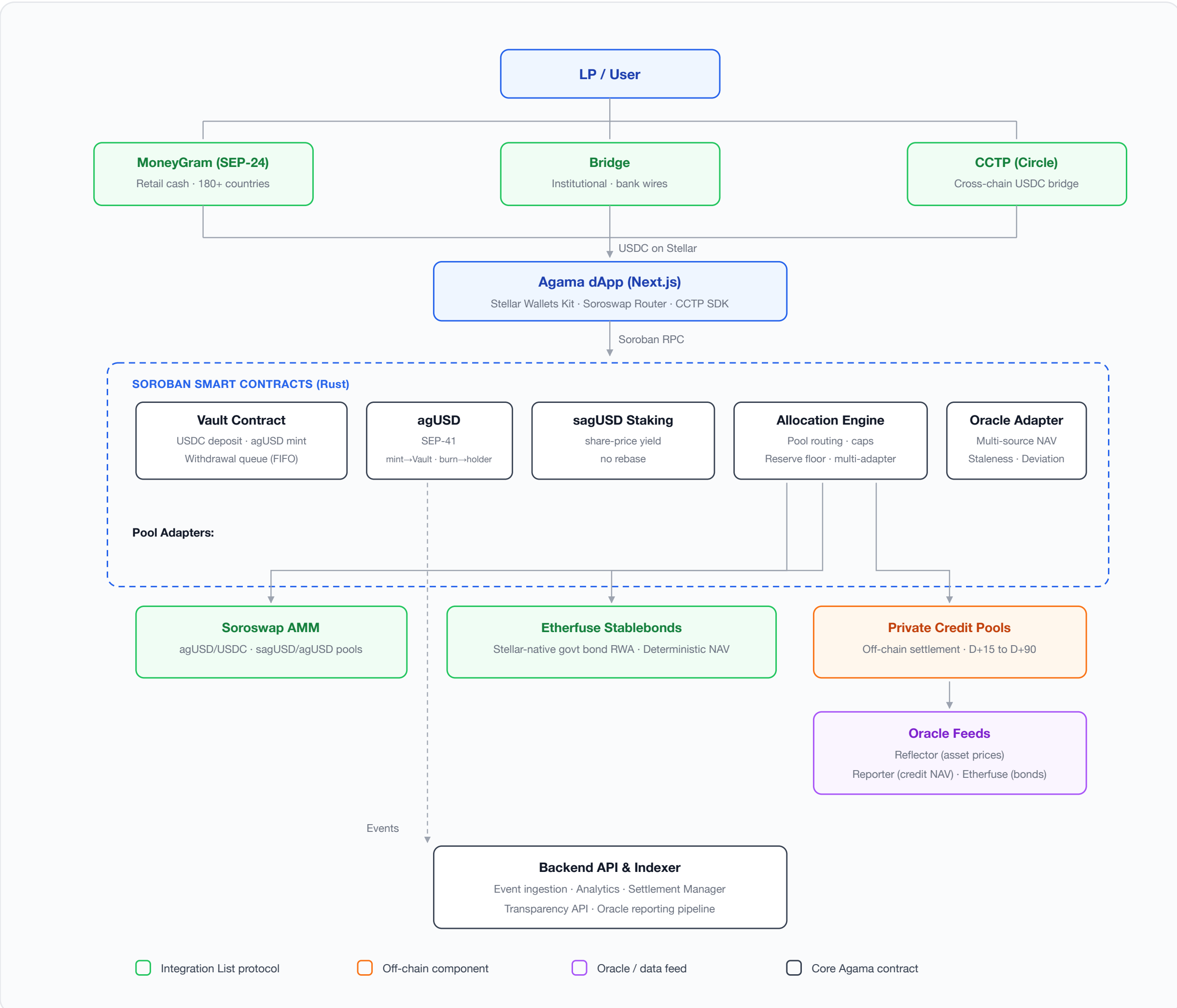
2.4 Data Flows

1. User converts fiat → USDC via MoneyGram (SEP-24) or Bridge, or bridges USDC from Ethereum via CCTP, or acquires USDC on Soroswap / Stellar DEX.
2. User connects wallet (Freighter, xBull, Albedo) via Stellar Wallets Kit.
3. User deposits USDC into Vault Contract → receives agUSD 1:1.
4. User optionally stakes agUSD → sagUSD at current exchange rate.
5. Curator whitelists pools; Allocation Engine deploys capital:
   - Etherfuse Stablebonds → Stellar-native government bond yield
   - Private credit pools → off-chain originator allocations
6. Yield flows back: Etherfuse (bond interest), private credit (originator repayment → settlement → USDC on-chain).
7. Oracle Adapter receives NAV updates: Reflector for asset prices, Backend reporter for credit NAV, Etherfuse feed for bond pricing.
8. `distribute_yield()` increases sagUSD/agUSD exchange rate → sagUSD holders earn yield passively.
9. Backend Indexer captures all events → powers transparency dashboard + API.
10. User exits: `request_unstake()` then `claim()` after the cooldown returns sagUSD → agUSD; `request_withdrawal()` then `claim_withdrawal()` redeems agUSD → USDC through the Vault queue. Or swap on Soroswap without queueing. Off-ramp via MoneyGram or Bridge.

2.5 System Characteristics

- **Transparent** — All vault accounting, caps, and yield distribution enforced on-chain.
- **Non-Custodial Token Layer** — Users hold agUSD/sagUSD in their own wallets.
- **Risk-Constrained by Design** — Concentration caps checked at allocation time.
- **Composable** — SEP-41 tokens usable across Soroban protocols. sagUSD priced by a single exchange rate.
- **Ecosystem-Native** — Built on Soroswap, Etherfuse, CCTP rather than standalone.
- **High-Frequency Yield** — Stellar's sub-cent fees enable frequent on-chain distribution.
- **Secure by Design** — Admin-gated functions, pausability, oracle guards, STRIDE-modeled threats.
- **Open Source** — All Soroban contracts Apache-2.0 at mainnet launch.

System Architecture Diagram



3. Ecosystem Integrations

Agama builds on proven Stellar ecosystem protocols drawn from the **SCF Integration List**. Each integration serves a specific architectural role and replaces or augments a component that would otherwise be built from scratch.

3.1 DeFindex — Shared Yield Accounting Convention

Role: sagUSD follows the same economic convention as DeFindex. `distribute_yield()` raises the assets behind each share rather than minting shares or rebasing balances, so a position is valued from a single exchange rate and there is no claim step.

What this is not. It is a shared economic model, not call-level compatibility. DeFindex's own vault interface publishes neither `distribute_yield` nor `exchange_rate`. It is multi-asset, `get_asset_amounts_per_shares` returns one amount per underlying asset rather than a scalar price per share, and it exposes no vault-level yield distribution entry point. Agama routes no funds through DeFindex vault contracts and depends on no DeFindex deployment, and a DeFindex-integrated wallet would need integration work to read sagUSD. What is genuinely shared is the economics: shares are never rebased, nothing is pushed to holders, and a position appreciates because the assets behind each share grow. Verified against `vault/src/interface.rs` in `defindex-io/stellar-contracts`, the live repository, in September 2026; the former `paltalabs/defindex` was archived in July 2026.

3.2 Soroswap — AMM Liquidity

Role: Primary liquidity venue for agUSD/USDC and sagUSD/agUSD on Soroban. The dApp integrates Soroswap's Router API for swap routing across all Stellar liquidity sources.

Agama seeds pools at mainnet launch. The agUSD/USDC pool enables peg arbitrage: mint at 1:1 via Vault and sell on Soroswap if above peg, or buy on Soroswap and redeem if below.

> **Classic/Soroban constraint:** Soroban transactions cannot include Classic operations (path payments, DEX offers) in the same transaction. Soroswap swaps are Soroban-native and composable with contract calls. Classic DEX trades require a separate transaction. Agama does not claim atomicity between them. See Section 9.

3.3 CCTP — Cross-Chain USDC Bridge

Role: Native 1:1 USDC bridging from Ethereum/Arbitrum/Base. No wrapped tokens, no third-party bridge risk.

```
LP on Ethereum
    ├── Initiates CCTP burn (USDC burned on source chain)
    ├── Circle attestation service confirms burn
    └── LP mints USDC on Stellar
            └── Deposits into Agama Vault → agUSD minted
```

The dApp includes a "Bridge" tab using Circle's Bridge Kit SDK.

3.4 Etherfuse — Stablebonds as Native RWA Collateral

Role: First-day RWA collateral on Stellar. Etherfuse Stablebonds are Stellar-native tokens backed by government bonds with embedded yield.

Strategic value:

- Live vault at mainnet launch without depending on off-chain credit tokenization.
- Low-risk base yield layer (government bonds) complementing higher-yield private credit.
- Proof that the Allocation Engine generalizes across RWA types.

Oracle: Stablebond NAV is deterministic (public bond pricing). Staleness threshold relaxed to 48h.

3.5 Bridge / MoneyGram — Fiat On-Ramp

MoneyGram (SEP-24) — Retail cash on/off-ramp in 180+ countries via interactive anchor flows. Integration at the Backend level via SEP-24 adapter.

Bridge — Multi-currency institutional ramp (bank wires, ACH). Backend integrates Bridge REST API for payout initiation and webhook callbacks.

4. Contract Overview

4.1 Vault Contract

Purpose: Entry point for capital. Accepts USDC deposits, mints agUSD 1:1, manages NAV-based accounting and the FIFO withdrawal queue.

Key Functions

Function	Description
`__constructor(admin, usdc_token)`	Runs inside the deploy transaction, so there is no window between deployment and wiring for somebody else's `initialize` to land first. It takes USDC, which it checks answers the token interface, and deliberately not agUSD or the Engine: each of those is built against this Vault's address and cannot exist before it does, so each arrives afterwards through a setter that interrogates it.
`deposit(from, amount) → i128`	Transfers USDC, mints agUSD. Returns minted amount.
`request_withdrawal(from, amount) → u64`	Burns agUSD, enqueues claim. Returns claim_id.
`claim_withdrawal(from, claim_id)`	Pays USDC when Ready. FIFO order, or out of order for a claim the queue has already deferred. Fails with `PaymentRejected` rather than trapping if the token refuses to deliver.
`settle_withdrawal()`	Pays the head claim to its recorded owner. Permissionless: no claim id, no recipient, so the caller can neither redirect nor reorder. If the token refuses to deliver, the claim is marked deferred and stepped over, unpaid and still owed.
`is_deferred(claim_id) → bool`	Whether the queue stepped over this claim because it could not be delivered. Its owner collects it through `claim_withdrawal` whenever the obstruction is gone.
`set_reserve_floor(admin, floor_bps)`	The Vault's own copy of the floor, enforced in `settle_allocation`. Ships closed at 10000.
`record_repayment(amount)`	Engine-called. The Vault verifies the cash arrived in its own balance before reducing `deployed_capital`.
`record_writedown(admin, amount)`	Engine-called and admin-signed. Reduces `deployed_capital` with no cash arriving, and raises `recognised_losses` by the same amount so the floor's base does not move.
`record_recovery(admin, amount)`	Engine-called and admin-signed. Books capital that had been written off and has come back, and releases the recognised loss against it. Refuses any amount the Vault cannot see arriving in its own balance, and the loss it releases is matched stroop for stroop by cash entering free reserves — so `floor_base` does not move here either. What a write-down cannot buy, a recovery cannot buy back.

Function	Description
`bump_claim(claim_id)`	Postpones the archival of a claim record. Permissionless: it cannot shorten a TTL, cannot alter a claim, and the caller pays the rent, so extending a stranger's claim is a donation rather than an attack. It exists because claims are bumped only when they are written, and a claim waiting behind a queue that is allowed to stall is a claim nothing writes to.
`free_reserves() / outstanding_liabilities() / get_net_assets() / deployed_capital()`	The four numbers the book is computed from, all readable by anyone.
`recognised_losses() → i128`	Deployed capital written off and not recovered. Not an asset, and `get_net_assets()` correctly excludes it. It rises on a write-down and falls in exactly one way, `record_recovery`, which requires the cash to have arrived.
`floor_base() → i128`	The denominator the reserve floor is a share of: net assets plus `recognised_losses()`.
`propose_admin(admin, new_admin) / accept_admin(new_admin)`	Two-step admin handover.
`settle_allocation(pool, amount)`	Releases idle USDC to a pool. Callable only by the Allocation Engine, which has already checked the caps and the floor.
`set_agusd(admin, agusd_token)`	Points the Vault at the token it mints, and the only door that pointer has. The token must name this Vault as its minter — the first Vault ever deployed here pointed at one that exposed no `mint` at all and could never issue a unit — and the pointer closes at the first deposit.
`set_engine(admin, allocation_engine)`	Repoints the Engine allowed to release reserves. Refuses any address that does not answer that it governs this Vault with an empty book, and refuses to move at all while this Vault has capital deployed.
`set_oracle(admin, oracle, feed_id)`	Points the Vault at an Oracle Adapter and the feed it reads NAV from.
`set_paused(admin, paused)`	Circuit breaker.
`idle_reserves() → i128`	USDC the Vault is holding. What the reserve floor protects and what claims are paid from.
`get_total_assets() → i128`	Gross: idle reserves + deployed allocations. Counts USDC owed to the withdrawal queue, which is still an asset until it is paid. `get_net_assets()` is the figure the caps and the floor use.
`get_nav() → i128`	Latest validated NAV from Oracle Adapter. Propagates `OracleStale` rather than returning an old number.
`get_claim(claim_id) → Claim`	The stored claim record.

Function	Description
`claim_status(claim_id) → ClaimStatus`	Pending, Ready or Claimed. Ready is computed, not stored: a claim becomes payable when the queue reaches it and reserves cover it, without anyone touching it.
`queue_head() → u64`	Next claim id that may be paid.
`queue_tail() → u64`	Next claim id to be handed out.
`queue_length() → u64`	Claims requested and not yet paid.
`deposits() → u64`	Deposits taken since deployment. What `set_agusd` keys off.

Storage

Instance: Admin, PendingAdmin, UsdcToken, AgusdToken, AllocationEngine, Oracle, OracleFeed, Paused, QueueHead, QueueTail, Deposits, FloorBps, Deployed, Queued, Booked, WrittenOff.
Persistent: Withdrawal claims (keyed by claim_id), and a deferred flag per claim id for the ones the queue has stepped over. Claim records carry a 90 day TTL, bumped whenever they are written and by `bump_claim` whenever anybody asks. That entry point is the difference between a documented keeper and a keeper with something to call: an archived persistent entry cannot be read at all, so a head claim that outlived its TTL used to stop the whole queue until an out-of-band `RestoreFootprint` put it back.

Events

`Deposit(user, amount, minted)` · `WithdrawalRequested(user, claim_id, amount, queue_position)` · `WithdrawalClaimed(user, claim_id, amount)` · `PauseToggled(paused)` · `AgUsdRepointed(agusd)` · `EngineRepointed(engine)` · `ReserveFloorSet(floor_bps)` · `RepaymentRecorded(amount, deployed)` · `WriteDownRecorded(amount, deployed, recognised_losses)` · `RecoveryRecorded(amount, applied_to_losses, recognised_losses)` · `WithdrawalDeferred(user, claim_id, amount)` · `AdminProposed(new_admin)` · `AdminChanged(admin)`

Security

Constructor rather than a front-runnable `initialize` · `require_auth()` on all state-changing calls · Zero/negative validation · Pause circuit breaker on deposits, requests and allocations, never on payouts · Minimum withdrawal amount · FIFO queue (no priority, no stalling, and no freezing on a payout the token refuses) · The reserve floor enforced against the Vault's own book, so no Allocation Engine can take reserves below it, and against a base a write-down cannot move, so no admin can either · Two-step admin handover.

4.2 agUSD Token Contract (SEP-41)

Purpose: Composable synthetic dollar. `mint` is restricted to the recorded minter, which is the Vault Contract address: it is the only address that can bring agUSD into existence, and `set_minter` stops working at the first mint, so every unit in circulation was created by the minter named in the deployment record.

`burn` and `burn_from` are **not** minter-gated. They are the standard SEP-41 holder-authorized paths: any holder can burn their own agUSD, and a spender can burn against an allowance. The Vault's `request_withdrawal` uses exactly that path, calling `burn` on the withdrawer in a transaction the withdrawer has already signed, rather than a privilege of its own. Supply can therefore only go up through the Vault, and can go down through anyone holding the token — which is the correct asymmetry for a redeemable synthetic dollar, since burning agUSD destroys a claim rather than creating one.

Standard SEP-41 interface: `transfer`, `transfer_from`, `approve`, `allowance`, `balance`, `burn`, `burn_from`, `decimals`, `name`, `symbol`, `total_supply`.

Events: `mint` · `burn` · `transfer` · `approve` (SEP-41) · `MinterSet(minter)`

4.3 sagUSD Staking Contract

Purpose: Yield-bearing staked agUSD. Share-based vault accounting on the same share-price economics as DeFindex, not its contract interface. Yield increases the sagUSD/agUSD exchange rate.

Key Functions

Function	Description
`stake(from, agusd_amount) → i128`	Locks agUSD, mints sagUSD shares at the current rate. Returns shares minted.
`request_unstake(from, shares) → i128`	Step 1 of 2. Burns the shares immediately, prices them at the current rate and locks the agUSD owed behind the cooldown. Returns the assets owed.
`claim(from) → i128`	Step 2 of 2. Pays out a matured unstake request. Reverts while the cooldown is still running.
`cooldown() → u64`	Seconds between a request and the moment it can be claimed. 60 on testnet.
`pending(addr) → Pending`	The caller's queued unstake: assets owed and the timestamp it becomes claimable.
`distribute_yield(amount)`	Deposits yield, increases assets-per-share. Authorized distributor only, which in V1 is the stored admin: the call takes no distributor argument and authorizes the recorded admin address, whose own agUSD is what moves.
`exchange_rate() → i128`	Current agUSD per sagUSD share (scaled to 7 decimals).
`share_price() → i128`	Alias of `exchange_rate()`, the name this contract shipped with. Same computation, retained because the generation 1 agUSD calls it on the credit vaults.
`nav() → i128`	Total agUSD the contract is accountable for. It moves only through `stake`, `request_unstake` and `distribute_yield`, all three backed by a transfer.
`total_shares() → i128`	sagUSD in circulation.

There is no NAV setter. The contract shipped with `report_nav(new_nav)`, admin-gated, accepting any non-negative value with no bound and no event, described as being for demo and reconciliation. NAV is the denominator of both directions of the share price — `stake` mints `amount * supply / nav`, `request_unstake` returns `shares * nav / supply` — so it was not a reporting convenience but an instruction to reprice every share in the contract. Against 1000 agUSD staked: `report_nav(1)`, stake 99 stroops and take 99% of the share supply, `report_nav` back, unstake, leave with 990 agUSD of somebody else's deposit. It has been removed rather than bounded, because `distribute_yield` already does the legitimate job and cannot overstate the book.

Unstaking is two steps, not one. There is no single `unstake()` call. `request_unstake` burns the shares at request time and prices them there, so a queued position cannot keep earning, be sold, or be re-requested while it waits; `claim` pays it out once `cooldown()` has elapsed. Pricing at request rather than at claim is what stops the cooldown being used as a free option on the exchange rate.

Events: `mint` · `burn` · `transfer` · `approve` (SEP-41, on the share token) · `AgUsdRepointed(agusd)`. Yield distribution is observable as the resulting change in `nav()` and `exchange_rate()` together with the underlying agUSD `transfer` into the contract; it does not currently emit a dedicated event of its own.

4.4 Allocation Engine Contract

Purpose: Routes vault capital across pool adapters (Etherfuse, private credit) with on-chain concentration cap enforcement and a reserve floor. All four limits are measured in basis points, so they are read in the same units and cannot be compared wrongly. The three caps are a share of net assets; the floor is a share of `floor_base`, which is net assets plus everything written off and not recovered. The difference is deliberate and it only ever runs one way: a larger base makes the floor *tighter* and would make a cap *looser*, so the write-off term belongs in one denominator and not in the other.

Write-offs reach the caps through the numerator instead. Every cap is measured on `charged_exposure`, which is what a pool holds plus what has been written off against it, and not on live exposure alone. Live exposure is a quantity `write_down` sets to zero while the adapter goes on holding the cash, so measuring against it let the same pool be filled to its cap, written off and filled again without limit: a pool capped at 40% could take everything the reserve floor would release, in slices that each read as inside the cap, with the originator and jurisdiction sums following it up because they are built from the same per-pool numbers. Charging the write-off is the numerator half of the fix the floor got in its denominator, and it is a product decision as much as a correction — a defaulted originator does not get its limit back by defaulting, and re-funding it takes an explicit decision rather than a side effect of recognising a loss.

Key Functions

Function	Description
`__constructor(admin, vault)`	Runs inside the deploy transaction, closing the window an `initialize` left open. The Vault must answer `admin()`, which an ordinary account cannot, and must answer with this Engine's own admin, because the calls that recognise and reverse a loss need one signature that satisfies both contracts. Ships fail-closed besides: every cap at zero and the reserve floor at 10000 bps, so an unconfigured Engine can deploy nothing.
`register_pool(admin, pool_id, originator, jurisdiction, cap_bps)`	Whitelists a pool with metadata and cap. Refuses any adapter that does not name this Engine and this Engine's Vault: an adapter pointed elsewhere takes capital from this Vault and repays a third party while `deallocate` decrements the book as though the money had come home.
`set_caps(admin, pool_cap_bps, originator_cap_bps, jurisdiction_cap_bps)`	Updates global concentration limits, in bps of net assets.
`set_reserve_floor(admin, floor_bps: u32)`	Sets the minimum **share of `floor_base`**, in basis points, that must stay as free USDC in the Vault. Rejects anything above 10000. Admin-gated; emits an event. The Vault keeps its own copy and enforces it independently.
`set_vault(admin, vault)`	Repoints the Engine at a different Vault, running the same interrogation the constructor runs, so a repair cannot leave the pair in a state the constructor would have refused to create. Refused while any capital is deployed, so the book and the balance sheet the caps measure it against stay one Vault's.
`allocate(admin, pool_id, amount)`	Deploys capital. Reverts if any cap is exceeded, or if the call would leave free reserves below `floor_bps` of `floor_base`. The Vault applies the same floor again when it releases.

Function	Description
`write_down(admin, pool_id, amount, reason)`	Recognises a credit loss. Reduces this Engine's exposure, the adapter's own, and the Vault's deployed capital, in one transaction, with no cash required, and raises `written_off` by the same amount so the floor's base does not move. The amount is also charged against the pool's concentration cap until the cash comes back. Admin-gated, and it refuses with `AdminMismatch` rather than trapping four frames down if the Engine's admin and the Vault's have diverged. Emits an event carrying the reason.
`recover(admin, pool_id) → i128`	Brings home whatever an adapter holds above its booked exposure: a written-off position that recovers, and interest paid above principal. Both used to be unreachable, because `deallocate` is capped at booked exposure and a written-off position has none, and because an adapter holding USDC cannot be repointed — one stroop of stranded cash closed the only repair path the adapter had, and it cost three redeployments before it was worth fixing. The caller aims nothing: the destination is the adapter's stored Vault and the amount is the surplus, neither of them a parameter.
`deallocate(pool_id, amount)`	Records repayments returning to the Vault, and tells the Vault, which verifies the cash arrived before its own book is allowed to fall.
`propose_admin(admin, new_admin)` / `accept_admin(new_admin)`	Two-step admin handover.
`get_exposure(pool_id) → i128`	Current allocation per pool.
`get_exposures() → Map`	Full allocation state. Pools with no exposure appear as zero, so the map doubles as the whitelist.
`total_allocated() → i128`	Total booked as deployed across every pool.
`caps() → Caps`	The three concentration limits currently in force, in bps.
`reserve_floor_bps() → u32`	Current reserve floor, in bps of `floor_base`. Readable by anyone.

Function	Description
`written_off() → i128`	Exposure written off and not recovered. Not an asset, and `total_allocated()` correctly excludes it.
`written_off_pool(pool_id) → i128`	Written off against one pool and not recovered. This is what a write-down costs that pool's cap.
`charged_exposure(pool_id) → i128`	The quantity the three caps are actually measured on: deployed plus written off. It differs from `get_exposure()` only after a write-down, and that difference is the whole reason a write-down can no longer reopen a cap.
`admin_aligned() → bool`	Whether this Engine's admin and the Vault's are the same address. It is false exactly when `write_down` and `recover` will refuse. Nothing can stop an operator rotating one key at a time, and nothing should — a rotation that needed a counterparty's cooperation would be one a hostile counterparty could block — but the divergence should be visible before an incident puts a number on it, and this is one call.
`floor_base() → i128`	The denominator the floor is a share of: the Vault's free reserves, plus what this Engine has booked as deployed, plus `written_off()`.
`get_reserve_ratio() → u32`	Free reserves as an actual share of `floor_base`, in bps: the number the floor is a lower bound on, read against the same base. Measured against net assets instead, this went *up* when a loss was recognised, which is the opposite of what losing money does to a liquidity position.
`get_pool(pool_id) → Pool`	A registered pool's originator, jurisdiction and cap.
`pools() → Vec<Address>`	Every registered pool adapter.

Adapter Interface

All pool types implement a uniform adapter interface. The Engine is agnostic to pool type. Adapters handle oracle queries, token transfers, and position encoding specific to each pool type.

Adapter	Underlying	Settlement	Oracle
Etherfuse	Stablebond contracts	Instant (on-chain)	Etherfuse feed (48h staleness)
Private Credit	Off-chain originator	D+15 to D+90	Custom reporter (7d staleness)

Operational Model

V1 (grant scope): Admin-directed allocation. The Curator calls `allocate()` manually. The Engine enforces constraints but does not decide autonomously.

V2 (post-grant): Off-chain optimizer computes target allocations and submits through the same admin-gated functions. Same cap enforcement, same governance guardrails.

4.5 Oracle Adapter

Purpose: Single source of truth for NAV data, bridging multiple feed types with unified validation.

Data Sources

Feed	Source	Trust Model	Staleness	Deviation	Band	Interval
Asset prices (USDC)	Reflector	Decentralized	1 hour	200 bps	0.90–1.10	5 min
Private credit NAV	Off-chain report → Backend → Reporter key	Centralized (V1, disclosed)	7 days	500 bps	0.50–2.00	1 hour
Etherfuse bond price	Etherfuse API / on-chain	Deterministic	48 hours	none	0.50–2.00	1 hour

The band and the interval exist because a deviation bound is relative and cannot cover two cases on its own. It cannot reach the **first** report for a feed, since a bound on a move needs something to move from: before the band, `push_nav(i128::MAX)` was accepted and became the reference every later bound was a percentage of. And it says nothing about **how many** reports there can be, so forty pushes of +5% moved a NAV sevenfold in forty seconds with every one of them inside the bound. The interval is measured in ledger time between accepted values rather than in reported timestamps, because the reporter chooses those and can submit forty of them, a day apart, in the same minute.

Pipeline

```
Originator (servicing data)
    → Agama Backend (reconciliation + validation)
        → Reporter key calls push_nav(nav, timestamp)
            → Oracle Adapter validates:
                ✓ Caller in authorized reporter set
                ✓ NAV inside the feed's absolute band (covers the first report)
                ✓ Timestamp > last update, and not ahead of ledger time
                ✓ Feed's minimum interval elapsed in ledger time
                ✓ Deviation ≤ 5% from previous NAV
                ✗ If exceeded → nav_rejected event
            → Vault Contract calls get_nav()
                → Reverts with OracleStale if feed older than threshold
```

Failure Modes

Failure	Impact	Mitigation
Reporter offline	Withdrawals/allocations revert	Deposits/stakes continue. Admin assigns backup reporter.
Reporter compromised	False NAV pushed	Deviation bounds reject. V2: multi-reporter quorum.
Originator misreports	Incorrect NAV	Backend reconciliation. >5% requires admin confirmation.
Reflector offline	Display-only impact	Core operations do not depend on Reflector.

Test Coverage (all contracts)

End-to-end flows (deposit → stake → yield → redeem) · Cap-violation rejection · Re-initialization guards · Access control, with targeted authorizations rather than a blanket mock · Zero/negative validation · Oracle staleness, deviation, band and rate limit · Withdrawal queue ordering and permissionless settlement · A hostile Allocation Engine bounded by the Vault's own floor · Write-down accounting across three books · Two-step admin handover · Fuzzing on accounting invariants.

5. Settlement & Off-Chain Bridge

Private credit instruments settle off-chain. Repayments flow through traditional banking rails before conversion to USDC on Stellar. This is the structural bridge between real-world credit and DeFi accounting.

Settlement Flow

```
Originator (fiat repayment: principal + interest)
    → Settlement Account (off-chain bank, Agama entity or custodian)
        → Fiat → USDC conversion (via Bridge API or MoneyGram)
            → Settlement Manager (backend)
                → deallocate(pool_adapter, amount) on Allocation Engine
                    → USDC returns to Vault idle reserves
                        → Withdrawal queue processed (FIFO)
```

Settlement Timing

Pool Type	Settlement	Notes
Etherfuse Stablebonds	Instant	On-chain redemption
Private credit (invoice)	D+15 to D+30	Originator payment terms
Private credit (venture)	D+30 to D+90	Longer-dated instruments

Default Handling

1. **Detection:** Backend flags the missed payment. The Oracle receives a reduced NAV on the pool's feed, within that feed's deviation bound and rate limit.
2. **Write-down:** `write_down(admin, pool_id, amount, reason)` on the Allocation Engine reduces the recorded exposure without requiring the cash back. It is admin-gated, it emits an event carrying a reason, and it moves three books in the same transaction so they cannot disagree: the Engine's exposure record, the adapter's own, and the Vault's `deployed_capital`.

The Vault leg additionally requires the Vault admin's signature, because reducing the Vault's deployed book with no cash arriving is the one move that would otherwise let an Engine reset the limit its releases are measured against.

3. **Why it has to exist:** without it a default could not be recognised on-chain at all. Exposure moved only through `allocate` and `deallocate`, and `deallocate` transfers the USDC to the Vault *before* it decrements the book. A defaulted originator leaves the adapter holding nothing, so the transfer panics, the deallocation reverts, and the exposure reports full face value for the life of the contract — leaving every reserve ratio derived from it overstated by exactly the size of the loss, and that ratio is the number the concentration caps and the reserve floor are measured against.
4. **What a write-down must not buy:** nothing. Two signatures were never going to be enough on their own, because both of them are the same key and a real loss and an invented one pass identically — the arithmetic has to be the thing that says no. So the amount is added to `written_off` on the Engine and `recognised_losses` on the Vault, both cumulative, neither ever reduced, and both inside the denominator of the reserve floor for the life of the contract. Without that, the floor was a share of net assets, a write-down lowers net assets, and every write-off handed back releasable headroom worth `floor_bps` of itself: allocate to the floor, write the position off, allocate to the new floor, and 999.9999999 of every 1000 USDC left a Vault holding a 25% floor while the adapter kept every dollar, with every individual call inside both floors. Keeping the loss in the base is also the more correct treatment under a genuine default, because agUSD is redeemed one for one, so a loss reduces the assets and not one stroop of what the Vault owes: a book in that state should hold *more* cash against its liabilities, not conclude that it may now lend out more. New deposits raise the base and release headroom in the ordinary way, so it fails closed without stranding the contract.
5. **Pool removal:** Admin delists the defaulting pool. Existing exposure runs off naturally.
6. **Recovery:** A partial repayment against a position that still has exposure is a normal `deallocate`. A recovery on a position already written down to zero is `recover(admin, pool_id)`, which sweeps whatever the adapter holds above its booked exposure to the Vault, and releases the recognised loss against it on both books. There is still no path that writes an exposure back *up*: the recovery lands as free reserves, not as redeployed capital, and `floor_base` does not move, because the loss it removes from the base is exactly the cash it adds. Before it existed, a recovery had nowhere to go — `deallocate` is capped at the exposure and there was none — so the money sat in the adapter, and because an adapter holding USDC cannot be repointed, a single stroop of it also bricked the adapter's only repair path. Three generations of the private credit adapter were retired for exactly that, each retirement recorded in `deployments/testnet.json`.

Where the loss lands, stated as it actually is. A write-down makes the loss visible and stops the reserve ratio lying. It does not distribute it, and nothing else in the contracts does either. agUSD is a synthetic dollar: `deposit` mints exactly `amount`, `claim_withdrawal` pays exactly `claim.amount`, and NAV is read on neither path, so there is no share price for a loss to flow through. The withdrawal queue is strictly first in, first out. Put together, a shortfall lands on whoever is at the back of the queue when the cash runs out. That is a first-mover advantage and it is a run incentive, and it is written down here rather than glossed.

> **Correction.** Earlier revisions of this document said that a NAV write-down makes the agUSD share price “decline proportionally”, with the loss “socialized across all agUSD holders”. Neither is true of these contracts, and neither was true when it was written. There is no mechanism that reduces what a queued claim is paid, and no mechanism that reduces agUSD supply against a loss. The paragraph above replaces it.

How losses should be allocated between agUSD and sagUSD holders is an open product decision. sagUSD is the yield-bearing layer and takes the upside; the symmetrical arrangement is for it to take the first loss, which is a tranching decision with legal and disclosure consequences attached. It has not been made. Putting a loss-socialisation scheme in a contract to make this section read better would be encoding an answer nobody has agreed to, so the contracts recognise losses and stop there.

Custody Model

Component	Custody	Controller
USDC idle in Vault	Soroban contract	Non-custodial
Etherfuse Stablebonds	Vault adapter	Non-custodial
Private credit allocations	Off-chain (originator)	Originator + legal agreements
Settlement fiat	Off-chain bank account	Agama entity (custodial)

> **Explicit trust assumption:** Private credit allocations involve custodial, off-chain components. Users are informed that this exposure carries counterparty risk (default, settlement delay, FX risk). This is fundamental to private credit and cannot be eliminated on-chain. Concentration caps limit exposure to any single originator. Etherfuse allocations are fully on-chain and non-custodial.

6. Withdrawal Queue

Two-step FIFO withdrawal queue:

Step 1 — Request: `request_withdrawal(from, amount) → claim_id`. Burns agUSD, creates persistent claim record {claim_id, from, usdc_amount, timestamp, Pending}. Joins back of FIFO queue.

Step 2 — Claim: `claim_withdrawal(from, claim_id)`. Checks status = Ready. Transfers USDC from Vault reserves. Updates to Claimed.

Step 2, without the claimant: `settle_withdrawal()`. Pays whichever claim is at the head of the queue to the owner recorded on it, and any address may call it. It takes no `claim_id` and no recipient, so the caller has no lever to redirect a payment or skip ahead: the only thing it can do is what the head claim's own owner could have done. Without it, the head advanced only when that owner returned, which gave the holder of the head a veto over everyone behind them that they could exercise by doing nothing at all — one claim at the 1 agUSD anti-dust minimum, never claimed, froze every withdrawal in the protocol for as long as its owner cared to wait.

Step 2, when the claimant cannot be paid at all. That covered the head claimant who *will not* come back. It did not cover the one who *cannot* be paid, which is worse, because their own return does not fix it either. Paying a claim is a token transfer, and USDC is a Stellar Asset Contract over a classic asset, so the transfer fails whenever the destination has no trustline for USDC, has had it frozen by the issuer, has a limit below the claim, or no longer exists. Any of those trapped the whole invocation, so the head pointer never moved and every withdrawal behind it stopped permanently, with no admin path around it because there deliberately is not one. It cost one USDC and a lowered trustline limit, and it also happens by accident the first time an issuer freezes a claimant.

Delivery is therefore attempted rather than assumed. If the token refuses, `settle_withdrawal` writes nothing about the payment, marks the claim **deferred**, advances the head over it and emits `WithdrawalDeferred`. The claim stays unpaid and stays counted in `outstanding_liabilities`, so the cash behind it stays reserved and undeployable, and its owner collects it through `claim_withdrawal` — out of head order, from the recorded owner and nobody else, once — whenever the obstruction is gone. Called by the owner, a refused delivery still fails the call, with `PaymentRejected` rather than a trap, because the owner is the one party who can fix a missing or frozen trustline and should be told so.

The trade this makes is real and belongs in the open: a deferred claim loses its place in the queue, so claims behind it may be paid first. That cost falls on the only party who can do anything about its cause, and the alternative is one unpayable claimant holding every other depositor hostage indefinitely.

Reserve Floor

The Allocation Engine enforces the reserve floor as a contract-level invariant, and it enforces it **as a share of net assets rather than as an amount of USDC**. `set_reserve_floor(admin, floor_bps)` takes basis points; `reserve_floor_bps()` returns them; `get_reserve_ratio()` returns what free reserves actually are as a share of net assets, in the same units, so the limit and the reality are read off the same scale. Testnet runs at **2500 bps, 25%**.

Net assets are free reserves plus everything booked as deployed, and free reserves are the Vault's idle USDC less what the withdrawal queue is owed. `allocate()` computes what the Vault would be left holding once the release settles and reverts if that is below `floor_bps` of the base, so the check happens in the same transaction as the transfer and a refused allocation moves no funds and books no exposure.

The base is not net assets, and the difference is the point. `floor_base` is net assets *plus cumulative recognised losses*, because `record_writedown` lowers net assets with no cash moving anywhere, so a floor measured against them is a floor whose absolute size its own caller can lower at will. Allocating to the floor and then writing the position off, over and over, walked the whole of the reserves out of a Vault a slice at a time with every individual call inside the limit. `written_off` on the Engine and `recognised_losses` on the Vault never fall, so the base is invariant under a write-down exactly as it is under an allocation, and that invariance is what makes the floor hold across calls rather than only within one. For a protocol that has never taken a loss the two numbers are identical, which is the ordinary case and the one the deployed configuration is sized against. The three concentration caps keep net assets as their denominator, because a larger base loosens a cap and tightens a floor, and the write-off term belongs only where it tightens.

Free, not gross. `request_withdrawal` burns the agUSD immediately and leaves the USDC in the Vault until the claim is paid, so between those two moments the money sits on the balance sheet and already belongs to somebody. It appeared in no on-chain quantity at all: not in agUSD supply, which had been burned, not in idle reserves, not in total assets, not in the reserve ratio. Deposit 1000, queue all 1000 for withdrawal, and the Engine would still deploy 400 while `get_reserve_ratio()` reported a healthy 6000 bps, leaving a claim that could not be paid. `Vault::outstanding_liabilities()` is now the running total, `Vault::free_reserves()` is idle reserves net of it, and `Vault::get_net_assets()` is free reserves plus deployed capital. Every limit that asks how much may be deployed reads those.

The Vault enforces the same floor, on its own numbers. `settle_allocation` used to release USDC on the Engine's say-so and check nothing itself, on the reasoning that duplicating the Engine's limits would mean two implementations that can disagree. The Engine, however, is simply an address the Vault authorizes, and `set_engine`'s guard — which asks an incoming Engine whether it governs this Vault — is answered correctly by any contract that stores one address and returns it. So a floor enforced only in the Engine is a floor that any contract holding that authorization can skip. The Vault therefore keeps its own `reserve_floor_bps` and its own `deployed_capital()` book, incremented by every release it performs and reduced only by a repayment it can verify in its own balance or by a write-down carrying the admin's signature. The base is invariant under an allocation and under a write-down, so the limit holds across repeated calls and not only within one: a hostile Engine gets the first release an honest one would have been allowed, and then gets nothing. An honest Engine never meets the check at all, because it applied the same arithmetic to the same book one call earlier.

Why a share and not a sum. This is what replaced the Blend v2 liquidity buffer, and the unit is the reason the replacement is stronger rather than merely different. Blend gave fast-exit liquidity by holding a withdrawable position in someone else's lending market, which is only as instant as that market's utilization on the day it is needed. The floor gives it by never letting the USDC leave the Vault at all, which has no such dependency. But had the floor been written as a fixed number of dollars it would have inherited a different weakness: it would be most of a small book and a rounding error in a large one, and it would need re-tuning by hand every time the protocol grew. Withdrawal pressure scales with the size of the book, so the liquidity guaranteed against it has to scale too. A ratio does that on its own.

A floor only binds if the caps can reach it. That is a property of the configuration, not of the code. Two pools capped at 30% each can deploy at most 60% between them, so a 20% floor could never be the reason an allocation is refused: 40% would stay idle whatever the operator did, the pool cap would fire first every time, and the floor would pass its own unit test while doing nothing on-chain. The deployed configuration is chosen the other way round — pool caps of 4000 bps each, summing to 8000, against a floor that releases 7500 — so there are states reachable by ordinary allocations in which every concentration cap is satisfied and the floor is the only limit refusing the call. That case is exercised on testnet as a submitted transaction carrying the Engine's own error code, not asserted in prose.

The floor is set by the Curator through an admin-gated call, and every change emits a `ReserveFloorUpdated(floor_bps)` event. The Engine ships fail-closed: `__constructor()` leaves every cap at zero and the floor at 10000 bps, so an Engine that has been deployed but not configured cannot deploy capital at all.

Liquidity Sources (priority order)

1. Idle USDC reserves in Vault
2. New deposits (increase idle reserves)
3. Etherfuse Stablebond redemption (on-chain, instant)
4. Private credit repayment (off-chain, D+15 to D+90)

Expected Wait Times

Scenario	Wait
Vault has idle reserves	~5 min (next keeper cycle)
Reserves depleted, Etherfuse available	Minutes
Reserves depleted, only private credit	Days to weeks

Safeguards

- Minimum withdrawal amount (anti-dust).
- Queue depth monitoring — backend alerts trigger proactive Etherfuse redemptions.
- Strict FIFO — no priority, no jumping, including admin.
- No stalling either — `settle_withdrawal()` is permissionless and pays the head claim to its recorded owner, so an absent claimant cannot hold the queue.
- And no freezing — a claim the USDC contract refuses to deliver, which for a Stellar Asset Contract means any claimant without a trustline, with a frozen one or with a limit below the claim, is deferred and stepped over rather than trapping the call. It stays unpaid, stays owed, stays counted, and is collected later by its owner out of head order.
- Queued claims are subtracted from free reserves, so the capital behind them cannot be deployed out from under them.
- Payouts are outside the circuit breaker — a queued claim has already burned its agUSD, so pausing it would leave the holder with neither the token nor the cash.
- No claim expiry by design, and now none by accident either. Claim records are persistent with a 90 day TTL that is bumped whenever the claim is written, which for a claim sitting behind a stalled queue is never; the book behind it settles at D+15 to D+90, so outliving the TTL is an ordinary event. An archived persistent entry cannot be read, so the head claim archiving used to stop every withdrawal until somebody paid to restore it. `bump_claim(claim_id)` is the permissionless entry point that keeps it readable, and it is what the keeper this document already described was supposed to be calling.

7. Security Model (STRIDE)

Category	Threat	Mitigation
Spoofing	Unauthorized agUSD mint	`mint` restricted to the recorded minter (the Vault), and `set_minter` closes at the first mint. `burn` is holder-authorized and cannot inflate supply. `require_auth()` throughout.
Spoofing	Fake oracle reporter	Authorized reporter set. `push_nav()` validates caller. Rotation requires admin + event.
Tampering	NAV manipulation	Three guards, not one: a per-push deviation bound (>5% rejected), an absolute band per feed that also covers the first report, and a minimum interval in ledger time between accepted values. The reference point is persistent, so it cannot expire out from under the checks that read it.
Tampering	Allocation to compromised pool	On-chain concentration caps (pool, originator, jurisdiction) and the reserve floor, all four in bps of net assets. `allocate()` reverts if any is exceeded. `register_pool` refuses an adapter that does not name this Engine and this Engine's Vault.
Tampering	Hostile or mis-wired Allocation Engine	The Vault enforces the reserve floor itself in `settle_allocation`, against its own deployed capital book, which no Engine can write to. `set_engine`'s interrogation of an incoming Engine catches mis-wiring and is not relied on for more than that.
Tampering	Exposure that no longer exists	`write_down` recognises a credit loss on-chain, admin-gated and evented, so the reserve ratio stops overstating the book by the size of the loss.
Tampering	An admin using a write-down to reset a concentration cap	Caps are measured on `charged_exposure`, deployed plus written off, so a write-off keeps consuming the pool's limit, and its originator's and its jurisdiction's, until `recover` brings the cash back. Filling a pool, writing it off and refilling it now fails the same cap the first allocation passed.
DoS	Capital stranded in an adapter after a write-down	`recover` sweeps the surplus over booked exposure to the adapter's stored Vault. No destination parameter and no amount parameter, so it cannot be aimed, and the Vault verifies the arrival before it books it.
DoS	A head claim archiving and stopping the queue	`bump_claim(claim_id)` is permissionless and extends a claim's TTL, so keeping the queue readable does not depend on any one keeper being alive.
Elev. of Privilege	Front-running the deploy to claim admin	Every protocol contract wires itself in a `__constructor` that runs inside the deploy transaction, so there is no uninitialized contract for a competing `initialize` to reach.
Elev. of Privilege	A write-down used to create room under the reserve floor	Recognised losses stay in `floor_base` permanently, so a write-down moves net assets and does not move the floor's denominator. Alternating `allocate` and `write_down` releases nothing an honest single allocation would not have.
Repudiation	Originator denies allocation	Soroban events on every `allocate` / `deallocate`. Indexed with block provenance.
Repudiation	Disputed yield	Every distribution moves real agUSD in, so it leaves a SEP-41 `transfer` event and a matching move in `nav()` and `exchange_rate()`. Fully reconstructable from the chain. A dedicated `yield_distributed` event is planned.

Category	Threat	Mitigation
Info Disclosure	LP position exposure	Public chain by design. No private data in contracts.
DoS	Withdrawal queue flood	Minimum amount + agUSD burn cost. TTL on claim records.
DoS	Withdrawal queue stall	`settle_withdrawal()` is permissionless and pays the head claim to its recorded owner, so a claimant who never returns cannot hold the queue behind them.
DoS	Withdrawal queue freeze on an undeliverable payout	Delivery is attempted rather than assumed. A claim the token refuses is deferred and stepped over, unpaid and still owed, so a claimant with no USDC trustline, a frozen one or a limit below the claim cannot stop everybody else's withdrawals.
DoS	Reserves lent out from under a queued claim	Queued liabilities are subtracted from free reserves and net assets before any cap or floor is computed.
DoS	Oracle starvation	Deposits/stakes continue. Only withdrawals/allocations revert. Admin updates reporter set.
Elev. of Privilege	Admin key compromise	Admin cannot transfer USDC directly. Only `allocate()` (cap-bound and floor-bound, at both the Engine and the Vault) or `pause()`. Multi-sig (2-of-3) planned.
Elev. of Privilege	Admin key lost or compromised, permanently	Every contract carries a two-step handover, `propose_admin` then `accept_admin`, with the successor authorizing the second step itself so the role cannot be handed to an address nobody controls. Before this there was no rotation anywhere and a lost key was unrecoverable.

Access Control

Role	V1 Holder	Permissions	Evolution
Admin	2-of-3 multi-sig	Pause, register pools, set caps, set the reserve floor in bps on both the Engine and the Vault, write down a defaulted exposure, repoint counterparties while the guards allow it, update reporters, propose a successor admin	Governance + 48h timelock
Reporter	Dedicated hot wallet	Push NAV to Oracle	Multi-reporter quorum (2-of-3)
Yield Distributor	= Admin in V1	`distribute_yield()`, which moves the distributor's own agUSD	Dedicated service key, then keeper network
Curator	= Admin in V1	Whitelist pools, risk params	Independent risk committee

The write-down entry in that table is worth reading precisely, because it is the most dangerous call in the system and the one whose bound is arithmetic rather than a signature. It reduces recorded exposure with nothing arriving, so it is the move an admin would reach for to make a book look solvent or to create room under the floor. It cannot do the second: recognised losses stay in the floor's denominator permanently, so alternating `allocate` with `write_down` releases nothing that a single honest allocation would not have released. It can still do the first, in the sense that an operator who writes off a healthy position is misreporting; what stops that is the multi-signature admin and the timelock in this table, and the event every write-down emits, not a check inside a contract. It also requires the same address to be the admin of both the Engine and the Vault, so a partial rotation of one key disables loss recognition rather than weakening it. That is the safe direction to fail in and it is not the discoverable one, so the Engine says so twice: `admin_aligned()` reports the divergence at any time, and `write_down` and `recover` refuse with a named `AdminMismatch` instead of trapping on the Vault's `NotAdmin` several frames down. Rotate the Vault and the Engine together, or expect the next default to be the thing that tells you.

Pausability: When paused, deposits, withdrawal *requests* and new allocations are blocked. Withdrawal *payouts* are not: `request_withdrawal` burns the agUSD as it queues the claim, so pausing the payout would leave the holder with neither the token nor the cash for as long as the switch stayed on. Stopping the flows that create new obligations is what a breaker is for; refusing to honour obligations already on the books is something else. Staking/unstaking continue. Oracle updates continue.

Admin rotation: `propose_admin(admin, new_admin)` records a successor and changes nothing; `accept_admin(new_admin)` moves the role and can only be called by the proposed address, authorizing for itself. Two steps rather than one because a single-call setter aimed at an address nobody controls produces the unrecoverable state rotation exists to fix, in one transaction, with no second chance. There is no cancel entry point: a proposal replaces any earlier one, a pending admin can do nothing until it accepts, and an admin withdrawing a proposal proposes itself.

Upgrade path: V1 contracts are immutable. Upgrades require redeployment + migration. V2 may introduce controlled upgrade proxy with timelock.

8. Soroban Storage Management

Contract	Type	Data	Rationale
Vault	Instance	Admin, pending admin, tokens, pause, queue pointers, reserve floor, deployed capital, queued liabilities, accounted balance	Small, every call, contract lifetime
Vault	Persistent	Withdrawal claims (by claim_id)	Claims pending for weeks
agUSD	Instance	Metadata, supply, Vault address	Standard token data
agUSD	Persistent	Balances, allowances	Long-lived user data
sagUSD	Instance	Admin, pending admin, staked token, NAV, cooldown, stake counter, display allocations	Every stake/unstake
sagUSD	Persistent	Share balances, pending unstake requests	Long-lived user data; a request outlives the shares that created it
Alloc. Engine	Instance	Admin, pending admin, Vault, caps, reserve floor in bps, pool registry	Config, every allocation
Alloc. Engine	Persistent	Per-pool exposure records	Persist across settlement
Oracle	Instance	Admin, pending admin, reporter set, per-feed guards	Configuration
Oracle	Persistent	Latest NAV, its reported timestamp and the ledger time it was accepted at	The reference every guard is measured against; an expired one would remove the monotonicity check, the deviation bound and the rate limit at once

Bump strategy: Instance storage bumped automatically on invocation. Persistent hot data (balances, active claims, exposures, oracle reference points) bumped on user interaction. Cold data (claimed withdrawals) archives naturally. Backend keeper handles periodic bumps for system-critical entries, through `Vault::bump_claim(claim_id)` for the withdrawal queue, which is permissionless so that the keeper is a convenience rather than a dependency — anybody can keep the head of the queue readable, and a claim only archives if nobody at all does. Nothing a guard reads lives in temporary storage: the oracle NAV point moved out of it precisely because an entry that can expire is a guard that can be waited out.

9. Classic/Soroban Transaction Constraint

Protocol constraint: A Stellar transaction containing `InvokeHostFunction` (Soroban) CANNOT include Classic operations (path payments, DEX offers) in the same transaction.

Path	Mechanism	Composable with contracts?
Soroban (primary)	Swaps via Soroswap Router — single Soroban tx	Yes (deposit + swap in same tx)
Classic (secondary)	Trading agUSD on Stellar DEX order book — separate Classic tx	No (separate transaction)

The dApp supports both paths but does not claim atomicity between Soroban and Classic operations.

10. RWA Token Standards

Token	Standard	Transfer Restrictions
agUSD / sagUSD	SEP-41	None — permissionless, composable
Future RWA pool tokens	SEP-57 (T-REX) candidate	Whitelist, freeze, forced transfer, KYC hooks

V1: RWA positions exist as Allocation Engine accounting entries + off-chain legal agreements. SEP-57 is referenced as the intended standard for future on-chain RWA tokenization. agUSD is the democratized access layer (anyone can hold it); underlying credit positions retain compliance controls.

11. Technology Stack

Layer	Technology
Frontend	TypeScript, Next.js/React, Stellar Wallets Kit, Tailwind CSS
Backend	Node.js, Express, Redis
Blockchain	Soroban (Rust → WASM), Stellar SDK (JS), Horizon API, Soroban RPC
Database	PostgreSQL
Dev Tools	Rust + Stellar CLI, Docker, CI pipelines
Testing	soroban-sdk testutils, cargo test, fuzzing on invariants
Infrastructure	AWS/GCP, Grafana, Sentry

Ecosystem Integrations

Integration	Role	Integration List
DeFindex	Share-price yield economics for sagUSD, no contract calls	Yes
Soroswap	AMM pools + Router API	Yes
Etherfuse	Stablebonds — native RWA collateral	Yes
CCTP (Circle)	Cross-chain USDC bridge	Yes
MoneyGram	Fiat cash ramp (SEP-24)	Yes
Bridge	Institutional fiat ramp	—
Reflector	Oracle price feeds	—
Circle USDC	Native deposit asset	—

Stellar Ecosystem Proposals

SEP	Usage
SEP-41	agUSD and sagUSD token interface
SEP-24	MoneyGram interactive deposit/withdrawal
SEP-57	Future RWA token compliance (T-REX)